Algorithms & Data Structures

Course syllabus, Hult International Business School, Professor Roger Hunt

I have left the term, dates, meeting times, room and office hours blank so that I can reuse this syllabus in any term. Every blank is marked in square brackets.

Course information

Item	Details
Course code	[Course code to be added]
Credit hours	[Credit hours to be added]
Term	[Term to be added]. The term runs for 15 weeks.
Dates	[Dates to be added]
Meeting days and times	[Days and times to be added]. We meet twice a week, for 30 classes in all.
Campus and room	[Campus to be added] / [Room to be added]
Instructor	Professor Roger Hunt
Email	[Email to be added]
Office hours	[Office hours to be added]
Prerequisites	[Prerequisites to be added]
Course website	myCourses, Hult's learning platform: [link to be added]

What this course is

In this course you learn the critical algorithms and data structures of computer science, and every class works on them in two ways. In the first half of each class you solve two problems from LeetCode, a public website of programming problems, at the chalkboard: first an easier problem and then a harder one. You write each solution in Python, in chalk, without a computer or a phone, and you state how its running time grows with the size of its input, how much extra memory it needs, and roughly how many bytes of memory it uses in total. The problems start at LeetCode's Easy level. The first Medium problem arrives in week 4, and the first Hard problem in week 12.

In the second half of each class you put the algorithms and data structures into practice. You join live projects that I am working on and help build them, using Java, Python, Cursor and Claude Code. Here you write real code on a computer, with AI tools, and you see the structures and algorithms from the board at work in real projects.

Each class builds on the classes before it, and together the 30 classes cover the critical algorithms and data structures. The first nine classes give you the tools for measuring a solution, Big-O, Big-Omega and Big-Theta notation and the byte count, and then the basic structures that most programs use: arrays and dynamic arrays, strings, hash tables, stacks, queues and linked lists, together with the two-pointer and sliding-window techniques for scanning them, and recursion with divide and conquer. The next seven classes cover sorting, from merge sort to quicksort and counting sort, then binary search, trees, binary search trees, balanced search

trees and heaps. The last fourteen classes begin with graphs: searching them, ordering tasks that depend on each other, finding shortest paths with the algorithms of Dijkstra, of Bellman and Ford, and of Floyd and Warshall, union-find, and minimum spanning trees. They go on to backtracking and the limits of efficient computation, tries, greedy algorithms and dynamic programming, and the final class mixes every pattern without telling you which one to use.

A word that this course uses constantly is *pattern*. A pattern is a family of problems that share one solution idea. “Two pointers” is a pattern, for example: you keep two positions in a list and move them toward each other or in the same direction. Most new problems turn out to belong to a pattern you have already seen, and much of the skill this course builds is recognizing which one.

By the end of the term you should be able to read a new problem, recognize its pattern, choose a data structure that fits, plan your solution before you write code, measure what your solution costs in time and in memory, and build the structure in working code. The board work uses Python only and is done by hand: you write Python in chalk and then copy it onto a paper answer sheet. The project work in the second half uses Java and Python with AI coding tools.

What you will be able to do

By the end of the course, you will be able to do the following ten things.

1. Work out the running time and the auxiliary space of iterative and recursive Python code in Big-O, Big-Omega and Big-Theta notation, and explain your answer in writing and out loud. These notations describe how the work a program does grows as its input grows: Big-O gives an upper bound on that growth, Big-Omega gives a lower bound, and Big-Theta gives both, so a solution that is $\Theta(n)$ takes about twice as long on an input twice as large, and one that is $\Theta(n^2)$ takes about four times as long. Auxiliary space is the extra memory a solution uses beyond its input. Your analysis will include recurrences, the equations that describe the cost of a recursive algorithm, and amortized cost, which is the average cost of an operation over a long run of operations. Appending to a Python list is the standard example: an append is occasionally slow, when the list has to grow, but it is fast on average.
2. Estimate how many bytes a Python solution uses for a given input size, using the sizes of the objects that CPython, the standard Python interpreter, creates: integers, strings, lists, dictionaries, sets, deques, the node objects of linked lists and trees, and the frames of function calls.
3. Recognize which pattern a new problem belongs to, and defend your choice by comparing what each possible approach would cost in time and memory.
4. Write correct Python solutions by hand to Easy, Medium and Hard problems.
5. Build dynamic arrays, linked lists, stacks, queues, hash tables, heaps, binary search trees, balanced search trees, tries, union-find structures and graph representations, both by hand and in working code.
6. Apply the core algorithms of the course and trace them by hand. These are insertion sort, merge sort, quicksort, heap sort and counting sort; binary search; tree and graph traversal; topological sort; the shortest-path algorithms of Dijkstra, Bellman-Ford and Floyd-Warshall; the minimum-spanning-tree algorithms of Prim and Kruskal; backtracking; greedy methods; and dynamic programming.
7. Plan before you code. Before you write any code, you write down a brute-force approach, the pattern you will use, the steps in plain English, the cost you expect, and the edge cases your solution must handle.
8. Find correctness bugs and hidden inefficiencies in code that an AI tool has written, by reading and tracing the code, and correct them.

9. Explain what the complexity classes P and NP are, and recognize when a problem probably has no efficient algorithm, so that a careful search or an approximation is the realistic choice.
10. Apply these algorithms and data structures in real software, working in Java and Python with AI coding tools on live projects.

How the course works

This course runs on preparation and practice. I do not lecture. You learn each pattern before class. In the first half of each class you solve problems at the board, and in the second half you put the material into practice on live projects.

Before each class: the take-home

At the end of each class I hand out the take-home for the next class. The take-home for Class 1 is posted before the term starts. Each take-home names the pattern you will meet in the next class, assigns a reading from the free course textbook, lists two or three LeetCode practice problems of that pattern, and asks you to write a short plan for one of them.

Plan to spend at least two hours on each take-home. Read the assigned section, study the pattern, and attempt the practice problems on LeetCode. You may use AI tools to learn, and I expect you to.

Bring a written plan for one practice problem to class, on paper. The plan has six parts: the problem in your own words; the brute-force approach and its Big-O; the pattern you chose and why it fits; the steps in plain English; the time, auxiliary space and memory you expect; and two edge cases. An edge case is an unusual input that a correct solution must still handle, such as an empty list or a list with one element. The written plan is how you show that you arrived prepared.

First half of class: the board

Class starts with the work. When class begins you receive two problems. Problem A is the easier one and Problem B is the harder one. The take-home tells you the pattern ahead of time but not the problems themselves.

You solve each problem at the board, in Python, in chalk. Then you copy your solution onto the paper answer sheet I provide. For every solution you state three things: its time complexity, its auxiliary space, and an estimate of the total memory it uses in bytes, which you work out from the reference table later in this syllabus.

Phones and computers are locked away for the first half of class, and nobody writes code on a computer during it. The answer sheets include the table of memory sizes, so you do not need a device to make your estimates. You work on your own. If Hult's Accessibility Team has approved a device or another arrangement for you, that accommodation applies in this class; speak to me privately and we will set it up.

If you get stuck, ask me for help. Before you ask, write two attempts that did not work at the bottom of your answer sheet, because I will ask to see them. If you finish both problems correctly, I will give you an extension problem. [Format of the extension problems to be decided.]

Second half of class: live projects

In the second half of each class, devices come out. You join live projects that I am working on and help build them, using Java, Python, Cursor and Claude Code. This is where the algorithms and data structures from the board meet real code: you implement them, test them, measure them, and check the code that AI tools write, because those tools often produce code that runs but is wrong or slower than it claims. You register your attendance during the second half, when your devices are out. You retain authorship of the work you contribute.

Students who do excellent work are expected to contribute to a research paper that I submit to a journal, and they receive coauthor credit for that contribution. The section “Research and coauthorship track” below explains how this works.

What to write for every board problem

Item	What to write
Python solution	A plain function in readable Python. You may use anything built into Python, plus `deque` from the `collections` module and the `heapq` module where they fit naturally. Do not use any other library.
Time complexity	The Big-O of the running time, in terms of the size of the input. Name your variables: for example, n for the length of a list, m for the length of a second list, k for a number given in the problem, and V and E for the number of vertices and edges in a graph.
Auxiliary space	The Big-O of the extra memory your solution uses beyond the input. Anything your solution builds and returns counts.
Memory estimate	For the input size printed on the answer sheet, three numbers in bytes: the memory the input uses, the memory your solution’s own data uses, and the total. Write down the assumptions you made.
Failed attempts	If you asked for help, the two attempts you wrote down before you asked.

Python memory: reference sizes

Use the sizes in this table for your memory estimates. They come from CPython 3.13 on a 64-bit machine, measured with `sys.getsizeof` and `tracemalloc`.

Object	Approximate size
int	28 bytes for any integer below 2^{30}, which is about one billion, and 4 bytes more for each further 30 bits. Python creates the integers from −5 to 256 once and shares them, so those cost nothing extra.
float	24 bytes.
True, False and None	Python shares these objects, so they cost nothing extra. Only the 8-byte pointer to them counts.
str	41 bytes plus 1 byte for each character, when every character is plain ASCII. A string of one character is shared when that character is one of the first 256, which includes every ASCII character.
list of n items	56 bytes plus 8 bytes for each slot. A list made with `[0] * n` or `list(...)` has exactly n slots. A list built with `append` or with a comprehension keeps some empty slots in reserve, between none and about 13 percent extra, so allow about 6 percent on average. A list holds only pointers, so count each object it points to separately.
tuple of k items	40 bytes plus 8 bytes for each item: 56 bytes for a pair and 64 bytes for a triple.
dict	64 bytes when empty. With integer keys, about 37 bytes per entry at 1,000 entries, about 30 at 10,000 entries and about 52 at 100,000 entries; the cost per entry rises and falls because the table grows in steps. With string keys, about 26 bytes per entry at 1,000 entries and about 38 at 100,000. Count the keys and the values separately.

Object	Approximate size
set	216 bytes even when it holds a single element. About 33 bytes per element at 1,000 elements and about 42 bytes per element at 100,000. Count the elements separately.
deque	About 760 bytes when empty, plus about 8.3 bytes for each element.
node object, such as ListNode or TreeNode	About 90 to 100 bytes for the node and its pointers, plus 28 bytes for its value when the value lies outside −5 to 256.
recursive call	About 150 to 250 bytes for each call that has not yet returned, and more for a function with many local variables. Python stops a recursion at a depth of 1,000 unless you raise the limit.

A dict or a set grows by building a bigger table and copying itself into it, so for a moment it holds both tables. At large sizes, its peak memory is about 1.5 times the size of its final table. Lists and deques grow without such a jump. If you check an estimate with `tracemalloc`, expect it to report 32 bytes for each integer rather than 28, and expect it to leave out the memory of recursive calls, which it does not count.

Treat your result as an estimate. I accept an estimate within about 30 percent of the solution key’s figure, provided you write down your assumptions.

How the difficulty rises

Classes	Level of the two problems	What the classes cover
1 to 7	Easy and Easy	Big-O and memory, arrays, strings, hash tables, two pointers and sliding windows, stacks and queues
8 to 18	Easy and Medium	Linked lists, recursion, sorting, binary search, trees, balanced search trees, heaps and graph search
19 to 22	Medium and Medium	Topological sort, shortest paths and union-find
23 to 26	Medium and Hard	Minimum spanning trees, backtracking, tries and greedy algorithms
27 to 28	Medium and Medium	Dynamic programming starts again at Medium: first one-dimensional problems, then sequences
29 to 30	Medium then Hard, and Hard then Hard	Two-dimensional dynamic programming, then a final class whose problems carry no topic label

Class schedule

The course has 30 classes, two a week for 15 weeks. [Adjust this schedule if the term has a different length.] Every problem below is a real LeetCode problem, and the number and difficulty level are the ones that leetcode.com lists.

Week	Class	Pattern	Level	Problem A (easier)	Problem B (harder)
1	1	Big-O, Big-Omega, Big-Theta and Python memory	Easy / Easy	#1480 Running Sum of 1d Array	#217 Contains Duplicate

Week	Class	Pattern	Level	Problem A (easier)	Problem B (harder)
	2	Arrays and dynamic arrays	Easy / Easy	#283 Move Zeroes	#88 Merge Sorted Array
2	3	Strings and prefix sums	Easy / Easy	#14 Longest Common Prefix	#303 Range Sum Query - Immutable
	4	Hash tables	Easy / Easy	#242 Valid Anagram	#1 Two Sum
3	5	Two pointers and sliding windows	Easy / Easy	#125 Valid Palindrome	#643 Maximum Average Subarray I
	6	Stacks	Easy / Easy	#20 Valid Parentheses	#844 Backspace String Compare
4	7	Queues and deques	Easy / Easy	#933 Number of Recent Calls	#232 Implement Queue using Stacks
	8	Linked lists	Easy / Medium	#206 Reverse Linked List	#19 Remove Nth Node From End of List
5	9	Recursion and divide and conquer	Easy / Medium	#509 Fibonacci Number	#50 Pow(x, n)
	10	Sorting I: insertion sort and merge sort	Easy / Medium	#1331 Rank Transform of an Array	#912 Sort an Array
6	11	Sorting II: quicksort and counting sort	Easy / Medium	#1122 Relative Sort Array	#75 Sort Colors
	12	Binary search	Easy / Medium	#704 Binary Search	#34 Find First and Last Position of Element in Sorted Array
7	13	Trees and traversal	Easy / Medium	#104 Maximum Depth of Binary Tree	#102 Binary Tree Level Order Traversal
	14	Binary search trees	Easy / Medium	#700 Search in a Binary Search Tree	#98 Validate Binary Search Tree
8	15	Balanced search trees	Easy / Medium	#110 Balanced Binary Tree	#1382 Balance a Binary Search Tree
	16	Heaps, priority queues and heap sort	Easy / Medium	#703 Kth Largest Element in a Stream	#347 Top K Frequent Elements
9	17	Graphs and depth-first search	Easy / Medium	#733 Flood Fill	#200 Number of Islands
	18	Breadth-first search	Easy / Medium	#1971 Find if Path Exists in Graph	#994 Rotting Oranges

Week	Class	Pattern	Level	Problem A (easier)	Problem B (harder)
10	19	Topological sort	Medium / Medium	#207 Course Schedule	#210 Course Schedule II
	20	Shortest paths I: Dijkstra's algorithm	Medium / Medium	#743 Network Delay Time	#1631 Path With Minimum Effort
11	21	Shortest paths II: Bellman-Ford and Floyd-Warshall	Medium / Medium	#1334 Find the City With the Smallest Number of Neighbors at a Threshold Distance	#787 Cheapest Flights Within K Stops
	22	Union-find	Medium / Medium	#547 Number of Provinces	#684 Redundant Connection
12	23	Minimum spanning trees	Medium / Hard	#1584 Min Cost to Connect All Points	#1489 Find Critical and Pseudo-Critical Edges in Minimum Spanning Tree
	24	Backtracking, P and NP	Medium / Hard	#78 Subsets	#51 N-Queens
13	25	Tries	Medium / Hard	#208 Implement Trie (Prefix Tree)	#212 Word Search II
	26	Greedy algorithms and intervals	Medium / Hard	#55 Jump Game	#135 Candy
14	27	Dynamic programming: one dimension	Medium / Medium	#198 House Robber	#322 Coin Change
	28	Dynamic programming: sequences	Medium / Medium	#300 Longest Increasing Subsequence	#1143 Longest Common Subsequence
15	29	Dynamic programming: two-dimensional tables	Medium / Hard	#72 Edit Distance	#10 Regular Expression Matching
	30	Final class: choosing the pattern	Hard / Hard	#127 Word Ladder	#295 Find Median from Data Stream

I give out the problems in class. Each take-home tells you only the pattern, and its practice problems are different problems of the same type.

Grading

The rule

Every class counts toward your grade. If you are present and go to the board, you score 100 for that class, which is an A. If you miss a class, you score 0 for it. Your course grade is the average of your 30 class scores, converted to a letter grade on Hult's undergraduate scale.

Extra credit makes up for missed classes

A correct answer earns extra credit, and extra credit makes up for missed classes. Each correct board problem earns 50 points. An answer counts as correct when the solution works, its time and space analysis is right, and its memory estimate falls within about 30 percent of the key's figure. Every 100 points of extra credit turn one missed class from 0 into 100, so two correct answers make up one missed class. Extra credit only makes up for missed classes; it cannot raise your grade above 100. [Extra credit for extension problems to be set.]

For example, a student who misses two classes and answers four board problems correctly during the term has 200 points of extra credit. Those points turn both zeros into 100s, and the student's course grade is 100.

Extra credit makes up for an absence only in your grade for this course. Hult still counts the absence under its own attendance rules, which the section on Hult policies explains.

Using AI

You may use AI tools outside class and in the second half of class, and I expect you to use them. In the first half your devices are locked away, so the board work is yours alone.

Use AI to prepare for class. Ask it to explain the pattern, to walk you through examples, to quiz you, and to give you more practice problems. When you practice, write your own plan and your own attempt first. Only then use AI to check your plan and your solution and to find the point where your reasoning went wrong.

Treat whatever an AI tool gives you as a draft. AI tools often produce code that runs but gives wrong answers, or that is slower than the tool claims, so check every claim about complexity yourself, by hand. If I ask which tools you used and how you used them, be ready to show me.

The one requirement is that you can explain and defend your work in class, at the board and in the projects. If I ask why an answer is correct, how it works, or what you would change under different conditions, you need to answer on the spot. An answer you cannot explain does not count as a correct answer for extra credit.

Research and coauthorship track

Students who do excellent work in this course are expected to contribute to a paper that I submit to a peer-reviewed journal, and they receive coauthor credit for their contribution.

The signal I look for is consistently strong work, especially on the harder problems and the extension problems. I invite students directly. [Selection criteria and timing to be decided.]

Contributing can mean designing or running experiments and benchmarks, implementing and testing algorithms, analyzing results, reviewing the published research, or drafting sections of the paper. [Paper topic and target journal to be decided.]

You retain authorship of what you contribute. When your work goes into a paper, you are listed as a coauthor, and the work remains yours, whether you contributed it to a paper or to one of the live projects in the second half of class. Before any work on a paper begins, we agree in writing on what each person will

contribute and on the order of authors. [Details of the authorship agreement to be decided.] The work may continue after the term ends, through submission and revision. [Timeline to be decided.]

Tools

For the board work you write Python 3 and nothing else. For the live projects in the second half of class you use Java, Python, Cursor and Claude Code on your own laptop. [Licensing or access arrangements for Cursor and Claude Code to be added.]

To prepare outside class you need an AI assistant of your choice and a free LeetCode account for the practice problems. Python's `timeit` and `sys.getsizeof` let you check your own predictions about time and memory.

In the first half of class you need only chalk, the board, a pen and the paper answer sheets that I provide. Phones and computers stay locked away until the second half.

Materials

Required, and free

- Brad Miller and David Ranum, *Problem Solving with Algorithms and Data Structures using Python*, 3rd edition, free on Runestone Academy. This is the main textbook, and every take-home names the sections to read: https://runestone.academy/ns/books/published/pythonds3/index.html.
- Pat Morin, *Open Data Structures*, free online. Use it for the chapters on data structures. Its text is written in pseudocode, Java and C++ rather than Python: https://opendatastructures.org/.
- Jeff Erickson, *Algorithms*, free online. Use it for recursion, graphs, greedy algorithms and dynamic programming. It writes its algorithms in pseudocode: https://jeffe.cs.illinois.edu/teaching/algorithms/.
- MIT OpenCourseWare 6.006, *Introduction to Algorithms*, Spring 2020, with free lecture videos and notes. The course expects its students to program in Python 3: https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/.
- LeetCode, with a free account, for the practice problems in the take-homes and the problems we solve in class.
- The official Python documentation, especially the pages on built-in types, `collections`, `heapq`, `bisect` and `sys.getsizeof`: https://docs.python.org/3.13/.

Recommended

- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest and Clifford Stein, *Introduction to Algorithms*, 4th edition (MIT Press, 2022). This book, often called CLRS after its authors, is the standard reference.
- VisuAlgo, free interactive animations of data structures and algorithms: https://visualgo.net/en.

The textbook does not cover every pattern in this course. It has no sections on sliding windows, union-find, tries or intervals, for example. Where a take-home reaches one of those patterns, it says so and points you to another source.

Hult policies

Hult's official documents govern the policies below, and where this syllabus and an official Hult policy differ, the Hult policy applies. The references below are to the 2026–27 editions of Hult's *Academic*

Regulations and *Undergraduate Student Handbook*. Hult publishes new editions each year, so check the section numbers against the current edition.

Academic integrity

I expect you to do honest work and to represent it accurately. Using AI in the ways that the section “Using AI” describes is permitted and expected. Presenting work that you cannot explain, or misrepresenting who did the work, is not. Hult’s Honor Code and its rules on academic integrity are in sections 9.2 to 9.7 of the *Academic Regulations*. Those rules also require you to reference every use of a generative AI tool, using the APA citation format that Hult follows for all assignments (section 9.6.3).

Accessibility and accommodations

If you need an accommodation, contact Hult’s Accessibility Team at hultaccessibility@hult.edu and submit the Accommodation Request Form (https://hult.formstack.com/forms/accommodationrequest) as early as you can, because accommodations do not apply retroactively. Once your accommodation is approved, share it with me so that we can arrange it for this course. An approved accommodation applies in this class, including where it conflicts with the rule that devices are locked away. Section 8.4 of the *Undergraduate Student Handbook* describes the process.

Attendance

Attendance is central to this course, because all the work that counts happens in class. Hult’s own attendance rules also apply, and section 8 of the *Undergraduate Student Handbook* sets them out. Hult counts every absence, whether excused or not. A student who misses 30 percent of the scheduled sessions of a course is no longer eligible to have the participation component of the course graded and receives a formal warning. A student who misses 50 percent or more is no longer eligible to have any assessment graded, which will likely mean failing the course. Hult also requires you to register your attendance during each class session; in this course you register during the second half, when devices are out. The section “Grading” explains how absences affect your grade in this course.

Changes to this syllabus

I may adjust topics or pacing during the term. I will announce any change in class and on myCourses.